



ShopCrypt — Guide du TP de cryptographie

Messagerie chiffrée avec RSA-OAEP · HTML/CSS/JS pur · Web Crypto API · Vercel Functions · Neon PostgreSQL

Site en ligne : <https://shopcrypt-w3fq.vercel.app> · Code : github.com/Paul-Aymard/shopcrypt

Le site se met à jour automatiquement à chaque `git push` sur la branche `main`. Toutes les pages sont en ligne : boutique, connexion, clés, écrire, boîte de réception et démo RSA. Le style s'adapte au thème clair ou sombre.

1. Le principe en une image

Le **vendeur (A)** envoie un message confidentiel au **client (B)**. Seul B peut le lire, et **le serveur ne peut pas le lire non plus**.

① B génère ses clés

dans son navigateur. Il publie sa clé publique (cadenas ouvert) et garde sa clé privée (cle-privée.pem).



② A chiffre

son message avec la clé publique de B, dans son navigateur. Il obtient 256 octets illisibles, affichés en base64.



③ Le serveur stocke

seulement le chiffré base64 dans Neon. Il n'a aucune clé privée.



④ B déchiffre

avec sa clé privée, dans son navigateur. C, qui a une autre clé, échoue.

Analogie : la clé publique est un *cadenas ouvert* que tout le monde peut fermer. La clé privée est *la seule clé qui l'ouvre*.

2. Les choix cryptographiques (à savoir par cœur)

Paramètre	Valeur et justification
Algorithme	RSA-OAEP . OAEP ajoute un remplissage aléatoire : le même message donne un chiffré différent à chaque fois.
Taille de clé	2048 bits ($n = p \times q$ fait 256 octets). C'est le minimum recommandé aujourd'hui.
Hachage dans OAEP	SHA-256 , qui produit des empreintes de 32 octets.
Exposant public	$e = 65537 = 2^{16} + 1$. Il est premier et ne contient que deux « 1 » en binaire, ce qui rend le chiffrement rapide.
Limite 190 octets	256 (taille de la clé) $- 2 \times 32$ (deux empreintes SHA-256) $- 2$ (structure) = 190 . Formule : $k - 2 \cdot hLen - 2$. On compte des <u>octets</u> (TextEncoder) : « é » = 2 octets, « 🍌 » = 4 octets.
Chiffré	Toujours 256 octets , soit 344 caractères base64 . Le base64 transforme des octets en texte lisible.
Formats des clés	Publique : SPKI/PEM , envoyée à l'API. Privée : PKCS#8/PEM , téléchargée, <u>jamais envoyée</u> .

3. La sécurité côté serveur

- **bcrypt** (coût 10 + sel aléatoire) : le mot de passe n'est jamais stocké en clair.
- **JWT** signé en HS256 et valable 2 h : c'est le « badge » de connexion. Il est lisible par tous mais infalsifiable sans `JWT_SECRET`.
- L'identité vient **toujours du jeton**, jamais du corps de la requête.
- **Requêtes SQL paramétrées** : aucune injection SQL possible.
- Même message pour « pseudo inconnu » et « mauvais mot de passe » : on ne peut pas deviner quels comptes existent.
- `/api/keys` refuse tout ce qui n'est pas une clé publique RSA 2048 bits avec $e = 65537$.
- Les secrets sont dans des **variables d'environnement**, jamais dans le code.

4. Les fichiers du projet (les 10 étapes sont terminées)

#	Fichiers	Rôle	État
1	db/schema.sql	Tables <code>users</code> (avec <code>public_key</code>) et <code>messages</code> (avec <code>ciphertext</code>)	✓ fait
2	package.json, .gitignore, api/_lib.js	Dépendances, fichiers à ne pas publier, connexion Neon et vérification du JWT	✓ fait
3	api/register.js, api/login.js	Inscription et connexion (bcrypt + JWT)	✓ fait
4	js/api.js, login.html	Appels à l'API et session (sessionStorage), page de connexion	✓ fait
5	js/crypto.js	Génération, export et import des clés, chiffrement et déchiffrement RSA-OAEP, limite de 190	✓ fait
6	api/keys.js, keys.html	Générer la paire, télécharger le .pem, publier la clé publique, réimporter la clé privée	✓ fait
7	api/users.js, api/messages.js, write.html	Liste des destinataires, compteur d'octets, Chiffrer puis Envoyer	✓ fait
8	inbox.html	Cartes des messages, bouton Déchiffrer, erreur si mauvaise clé	✓ fait
9	demo-rsa.html	RSA codé à la main en BigInt (p, q, n, $\varphi(n)$, e, d, exponentiation modulaire)	✓ fait
10	css/style.css, index.html	Style, catalogue de 6 produits en FCFA, test complet, déploiement	✓ fait

5. Lancer le projet en local

1. `npm install`
2. Créer le fichier `.env` (et pas `.env.local` : `vercel dev` lit `.env`) avec `DATABASE_URL` (Neon → Connect) et `JWT_SECRET` (64 octets aléatoires).
3. `vercel dev`, puis ouvrir `http://localhost:3000/login.html`
4. Pour tester l'API avec PowerShell 5.1, lancer d'abord :
`[System.Net.ServicePointManager]::Expect100Continue = $false`

Ne jamais committer `.env`, `.env.local` ni `*.pem` : le `.gitignore` les bloque déjà.

6. Scénario de démonstration

1. **Kofi (client)** s'inscrit, génère ses clés, télécharge `cle-privée.pem` et publie sa clé publique.
2. **Awa (vendeuse)** écrit à Kofi. Le compteur indique par exemple 95/190 octets. Elle clique sur **Chiffrer** (le base64 s'affiche), puis sur **Envoyer**.
3. Montrer dans Neon que la colonne `ciphertext` est illisible.
4. Kofi ouvre sa boîte de réception et clique sur **Déchiffrer** : le message clair s'affiche.
5. **C** (ou Kofi avec un autre .pem) essaie de déchiffrer : l'erreur « *la clé ne correspond pas* » apparaît.

7. Questions probables de l'enseignant

Pourquoi ne pas chiffrer plus de 190 octets ?

RSA ne chiffre qu'un bloc plus petit que n , et OAEP en réserve 66 octets. Pour des messages longs, on utiliserait un chiffrement hybride (AES pour le message, RSA pour la clé AES), qui est hors du périmètre de ce TP.

Si la base Neon fuit, que lit le pirate ?

Des chiffrés, des clés publiques et des hachés bcrypt. Aucun message clair, car les clés privées ne sont jamais sur le serveur.

Pourquoi deux chiffrés différents pour le même message ?

OAEP mélange le message avec une graine aléatoire. On ne peut donc pas comparer des chiffrés pour deviner un message.

Le JWT est-il chiffré ?

Non, il est *signé*. Cela garantit son intégrité et son authenticité, pas sa confidentialité. La confidentialité des messages, c'est RSA-OAEP qui l'assure.

Pourquoi bcrypt plutôt que SHA-256 pour les mots de passe ?

SHA-256 est très rapide, donc facile à attaquer par force brute. bcrypt est lent, son coût est réglable, et il ajoute un sel.

Que se passe-t-il si Kofi perd son .pem ?

Ses anciens messages deviennent illisibles pour toujours. C'est le prix d'un vrai chiffrement de bout en bout.